Artificial Intelligence Technology and Application

Machine Learning Lab Guide

Student Version

Huawei Technologies CO., LTD.

Contents

1 Private Credit Default Prediction 1
1.1 Introduction 1
1.1.1 Objectives 1
1.1.2 Background 1
1.2 Procedure 2
1.2.1 Reading Data 2
1.2.2 Viewing Missing Values 2
1.2.3 Splitting the Dataset 3
1.2.4 Standardizing Data (Preprocessing Data) 4
1.2.5 Handling the Class Imbalance Issue (Preprocessing) 4
1.2.6 Performing Grid Search (Modeling) 4
1.2.7 Verifying Performance (Evaluation) 5
1.2.8 Saving the Model 6

1 Private Credit Default Prediction

1.1 Introduction

Under the impact of the Internet, financial institutions are suffering from internal and external troubles. On one hand, financial institutions encounter great competition and performance pressure from large financial and technology enterprises; on the other hand, more and more criminal groups use artificial intelligence (AI) technologies to increase the crime efficiency. These risk details are hidden in each transaction phase. If they are not prevented, losses will be irreparable. Therefore, financial institutions pose increasingly high requirements on risk management accuracy and approval efficiency.

This experiment will discuss the problem and perform practice step by step from the perspectives of problem statement, breakdown, priority ranking, solution design, key point analysis, and summary and suggestions, and cultivate the project implementation thinking and implement analysis of the private credit default prediction from scratch.

1.1.1 Objectives

Upon completion of this task, you will be able to:

- Understand the significance of credit default prediction.
- Master the development process of big data mining projects.
- Master the common algorithms for private credit default prediction.
- Understand the importance of data processing and feature engineering.
- Master the common methods for data preprocessing and feature engineering
- Master the algorithm principles of logistic regression and XGBoost, and understand the key parameters.

1.1.2 Background

The case in this document is for reference only. The actual procedure may vary. For details, see the corresponding product documents.

The company has just set up a project team for private credit default prediction. Engineer A was appointed as the offline development PM of the project. This project aims to:

- Identify high-risk customers efficiently and accurately using new technologies.
- Make risk modes data-based by using scientific methods.
- Provide objective risk measurement.
- Reduce subjective judgments.
- Improve risk management efficiency.

- Save labor costs.

The ultimate goal is to productize the results, so that front-end operating departments can identify transactions with credit default risks in a timely manner to avoid corporate losses.

1.2 Procedure

1.2.1 Reading Data

First, import the dataset. This document uses a third-party module from Pandas to import the dataset.

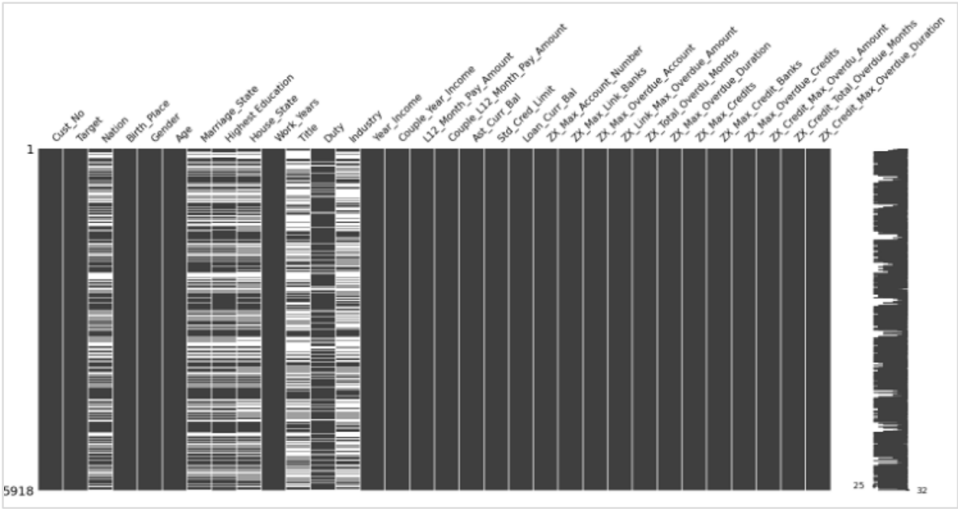
```
import pandas as pd
# Use pd.read_csv to read the dataset. (The dataset is stored in the current directory so that it can be read directly.)
# ./credit.csv indicates the current directory. The slash (/) here must be in the same direction as one in a directory of the Linux operating system (OS).
# In the Windows OS, the backslash (\) is used. Therefore, the slash in the file path must be the same as that in the Linux OS.
# Be aware of using the slash symbol in the same key on the keyboard as the question mark (?).


# An auxiliary module warnings can be imported.


# This module can help filter many redundant and annoying warnings.
# After data reading, some simple operations can be performed, for example:
# Run the following command to view all data.
data
# Run the following command to view the first 10 rows of data.
# Run the following command to view the length and width of data in the matrix format.
data.shape
```

1.2.2 Viewing Missing Values

```
# Check the data missing status in a visualized manner.
# The third-party library missingno is used.
```




```
# Many values are missing and need to be filled.
# There are many filling methods. The missing values can be filled with average values, medians, and the mode.
#The numeric types include discrete and continuous.
# If the average values are used, a new discrete value may be generated. Therefore, the mode is used


#isnull() is used to determine whether a value is null. If yes, True is returned. If not, False is returned.
# In Python, 1 is equal to True, and 0 is equal to False.
# Therefore, sum() is used for judgment. If the result is greater than 0, True is displayed.
# The features with missing values are placed in the missname list.
#fillna() is used to fill empty values with the mode.
```

1.2.3 Splitting the Dataset

Then, split the dataset. Before splitting the dataset, remove the index as it is unimportant and interferes with the model judgment, and remove **Target** (result) which cannot be used as input into the model.

```

```

X is equivalent to an independent variable in mathematics, and **y** is equivalent to a dependent variable. Import the dataset splitting function to split the dataset.

```
from sklearn.model_selection import train_test_split
```

X_train is the training set, and **y_train** is the answer to the training set. **X_test** is the test set, and **y_test** is the answer to the test set. **test_size=0.1** indicates that the ratio of the

training set to the test set is 9:1. **shuffle** indicates that the training set and test set are shuffled.

1.2.4 Standardizing Data (Preprocessing Data)

After the dataset is split, standardize the data.

```
from sklearn.preprocessing import StandardScaler
```

the standardization function **StandardScaler()** is first declared. The following **fit** function is used to obtain the standard deviation and average value of the dataset. Then, **transform** is used to transform the data.

1.2.5 Handling the Class Imbalance Issue (Preprocessing)

Next, handle the class imbalance issue, which is mainly about the difference between the number of positive results and the number of negative results of the data. In this dataset, the number of defaulters (represented by 1) is very small, and the number of non-defaulters (represented by 0) is very large. Therefore, the model tends to determine people as non-defaulters due to class imbalance. Check the current result ratio first.

```
from collections import Counter

# Use collections in the standard library to query the results.
# Import the third-party library imblearn.
from imblearn import over_sampling


# X_train_fix and y_train_fix are the corrected data.
# Next, check the number of samples.

# Check the corrected y_train_fix instead of the original y_train.
```

1.2.6 Performing Grid Search (Modeling)

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

# Declare the logistic regression algorithm and set max_iter (the maximum number of training times)
to 500.
# Perform judgment based on the cross verification thinking to help split the dataset.
# cv=5 indicates that the dataset is split into five equal parts.


# Apply grid search to find the optimal parameters through traversal.
# Import the grid search module.
from sklearn.model_selection import GridSearchCV
# C indicates the regularization coefficient.

# solvers indicates the optimization method.
```

```
# Combine the regularization coefficient with the optimization method using the dictionary method.

# Declare the logistic regression algorithm.

# Declare the grid search algorithm and describe the cross verification method.

# Perform training.

# Check the optimal accuracy.

# Check which parameters are optimal.
```

```
0.7691291714199306
{'C': 1.0, 'solver': 'lbfgs'}
```

1.2.7 Verifying Performance (Evaluation)

Then, verify the result.

```
from sklearn.metrics import accuracy_score,precision_score,recall_score,roc_auc_score
# Use the obtained optimal parameters for modeling.


# Construct a function to return the verification result.




# Output the performance data of the training set.

# Output the performance data of the test set.
```

```
accuracy score is:0.7029721955896453
precision score is:0.6881108939043895
recall score is:0.7424736337488015
aic:0.7029721955896452
accuracy score is:0.6537162162162162
precision score is:0.019230769230769232
recall score is:0.8
aic:0.7262350936967632
```

1.2.8 Saving the Model

Save the model.

```
# Import the joblib library
import joblib
# The dump function is used to save models. Enter the trained model and the model name into the dump function to save the model.
# The model file must be suffixed with .pkl.

# Load the saved model again.

# After the model is loaded, use the model for prediction directly.
```